A PROJECT REPORT ON

“CHAINSHIELD TRANSPORTATION SYSTEM”

Resilient Logistics & Dynamic Supply Chain Optimization

Submitted in Fulfillment of the Requirement
for Completion of Semester – 3 in
Master of Computer Application (MCA)
Academic Year 2026-27

To
LJ INSTITUTE OF COMPUTER APPLICATION
Affiliated to LJK University

Guided By:

Prof. Dhaval Shah

Prepared By: (Group No. 59)

Bhavik A. Visani (D51 - 25004401110170)
Diya Shah (D36 - 25003401110017)
Shruti Somani (D42 - 25004401110259)

DECLARATION

We, the undersigned students of Group No. 59, Master of Computer Application (MCA) Semester 3 at LJ Institute of Computer Application (affiliated to LJK University), hereby declare that the project entitled “ChainShield Transportation System” is our own original work, carried out under the guidance of Prof. Dhaval Shah, and has not been submitted, in whole or in part, for the award of any other degree, diploma, or certificate to this or any other institute or university.

We further declare that all the information, technical concepts, and design decisions presented in this report have been gathered, analysed, and implemented by us to the best of our understanding, and that any reference material used has been duly acknowledged in the Bibliography.

Date: _____ Place: Ahmedabad

Bhavik A. Visani (D51)

Diya Shah (D36)

Shruti Somani (D42)

CERTIFICATE

This is to certify that the project entitled “ChainShield Transportation System”, submitted for MCA Semester 3, has been carried out by Bhavik A. Visani (D51), Diya Shah (D36), and Shruti Somani (D42) of Group No. 59, under my guidance and supervision.

The work presented in this report is a bona fide record of the project carried out by the above-named students and, to the best of my knowledge, has not been submitted elsewhere for the award of any degree or diploma.

I wish them success in their future endeavours.

_____ _____

Prof. Dhaval Shah

Project Guide

Head of Department / Director

LJ Institute of Computer Application

PREFACE

Global trade today moves through supply chains of extraordinary scale and complexity — millions of shipments crossing ports, highways, railways, and air corridors every single day. The same forces that make this scale possible also make it fragile: a single storm, a congested port, or a mishandled hazardous cargo shipment can ripple into delays that cost businesses time, money, and customer trust. Historically, these disruptions have been discovered only after the fact, when a delivery has already slipped past its promised window.

This project, ChainShield Transportation System, was conceived to address exactly that gap. Rather than treating disruption management as a reactive, after-the-event exercise, ChainShield continuously evaluates every active shipment against a weighted risk model, flags the ones heading toward trouble, and proposes concrete rerouting alternatives before the delay materialises.

The idea originated from our participation in the Smart Supply Chains track of a solution-building initiative, where we built an initial prototype of ChainShield using FastAPI, React, and Firebase. For our MCA Semester 3 project, we chose to rebuild the same problem statement from scratch on a leaner, more traditional academic stack — Python (Flask), SQLite, and server-rendered Jinja2/Bootstrap templates, containerised with Docker — so that the system is simpler to explain, defend, and run consistently on any machine.

This report documents the problem definition, requirement analysis, system design, implementation, and testing of the rebuilt ChainShield Transportation System, prepared as part of the curriculum prescribed for MCA Semester 3 at LJ Institute of Computer Application.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us through the course of this project.

First and foremost, we thank LJK University and LJ Institute of Computer Application for providing us the opportunity and the academic environment to undertake this project as part of our MCA Semester 3 curriculum.

We are deeply grateful to our project guide, Prof. Dhaval Shah, for his valuable guidance, constructive feedback, and constant encouragement throughout the planning, design, and documentation of this project. His inputs have been instrumental in shaping the direction of ChainShield Transportation System.

We also extend our thanks to the Director and faculty of LJ Institute of Computer Application for their support, and to our classmates and peers who offered helpful suggestions during discussions on this project.

Finally, we thank our families for their patience and encouragement throughout this endeavour.

Thank you,

Bhavik A. Visani, Diya Shah, Shruti Somani

Group No. 59, MCA Semester 3

INDEX

Sr.	Particulars	Page No
1	**Introduction** 1) Problem Definition 2) Objective of the New System 3) Core Components 4) Project Profile 5) Need of the New System 6) Scope of the New System 7) Advantages and Limitations of the Proposed System 8) Proposed Time Line Chart 9) Targeted User	7
2	**Requirement Determination and Analysis** 1) Requirement Determination 2) Requirement Specification	12
3	**System Design** 1) Use Case Diagram 2) Data Dictionary 3) User Interface Design 4) Report Design	14
4	**Development** 1) Coding Standards 2) Implementation Details 3) Output Screens	19
5	**Testing** 1) Test Cases	30
6	**Proposed Enhancement**	34
7	**Conclusion**	35
8	**Bibliography**	36

1. INTRODUCTION

1.1 Problem Definition

Modern global supply chains manage millions of concurrent shipments across highly complex and inherently volatile transportation networks. Critical transit disruptions — ranging from sudden weather events to hidden operational bottlenecks such as port congestion, customs delays, or vehicle breakdowns — are chronically identified only after delivery timelines are already compromised.

In practice, logistics managers rely on manual tracking and reactive escalation: a disruption becomes visible only once a customer or downstream partner reports a missed delivery. By that point, the shipment is already late, alternate routing options have narrowed, and the cost of recovery — expedited freight, penalty clauses, damaged customer trust — has multiplied. There is no continuously running system that scores every active shipment's risk in real time and proposes a corrective route before the delay actually happens.

1.2 Objective of the New System

- Continuously analyse multifaceted transit data (weather, route congestion, cargo sensitivity, historic delay patterns) for every active shipment.
- Preemptively detect and flag potential disruptions before delivery timelines are compromised, using a composite, weighted risk-scoring model.
- Automatically generate optimised route-adjustment recommendations (hold, reroute via an alternate hub, or switch transport mode) the moment a shipment crosses a risk threshold.
- Notify the right stakeholder — manager, then the assigned field operator — the instant a decision is made, closing the loop between detection and execution.
- Provide role-based dashboards so admins, managers, and field operators (captains, pilots, drivers, loco pilots) each see only what is relevant to their responsibilities.
- Keep the system self-contained, inexpensive to run, and simple to demonstrate, using an open-source Python/Flask stack packaged with Docker.

1.3 Core Components

- Risk Engine — scores every active shipment on a recurring schedule using a 4-factor weighted model (weather severity, route congestion, cargo sensitivity, historic delay probability).
- Disruption Manager — converts a MEDIUM-or-higher risk score into a formal disruption record, escalates it to HIGH if the risk keeps rising, and triggers ranked route-solution generation.
- Route Solver — builds hold, hub-reroute and mode-switch alternatives using great-circle distances between hubs, and ranks them by a weighted score of residual risk, extra time and extra cost (₹).
- Notification & Chat Module — pushes real-time alerts over Flask-SocketIO to admins, managers and the assigned field operator, and supports direct messaging between users.

- Admin Panel & Reports — manages users, logistics hubs and cargo types, keeps an activity audit log, and provides reports with date-range filters and CSV export.
- AI Assistant — a conversational assistant (Google Gemini API, with an offline fallback engine) that answers natural-language questions using live shipment and disruption data.
- Live Map — a Leaflet.js visualisation of shipment positions, routes, hubs and shaded disruption zones that refreshes automatically after every risk scan.

1.4 Project Profile

Project Title	ChainShield Transportation System
Domain	Smart Supply Chains — Resilient Logistics & Dynamic Supply Chain Optimization
Project Description	A system that continuously scores active shipments for disruption risk and instantly proposes optimised route adjustments before delays cascade, with role-based dashboards for admins, managers, and field operators.
Front End	HTML5, CSS3, Bootstrap 5, Jinja2 templating, JavaScript, Leaflet.js, Chart.js
Back End	Python (Flask), Flask-SQLAlchemy, SQLite
Other Tools	Flask-Login, Flask-SocketIO, APScheduler, Flask-WTF, Google Gemini API, OpenWeatherMap API, Docker
Guide	Prof. Dhaval Shah
Submitted To	LJ Institute of Computer Application (LJK University)

1.5 Need of the New System

- Manual, reactive monitoring means disruptions surface only after a customer complaint or missed SLA — by then recovery options are limited and costly.
- Existing tracking dashboards typically show where a shipment IS, not what is about to go wrong with it — there is a visibility gap between tracking and prediction.
- Weather, congestion, cargo-type, and historical-delay signals sit in silos; no single continuously-running process fuses them into one actionable risk score per shipment.
- Rerouting decisions are usually ad hoc, based on a manager's personal experience, rather than a systematic comparison of ranked options with quantified cost, time, and risk trade-offs.
- Field operators (drivers, captains, pilots) often learn about a rerouting decision late — over phone or radio — introducing further delay and communication error.

1.6 Scope of the New System

Included in this project:

- Interval-based risk scoring for every active shipment (near-real-time).
- Automatic disruption creation once a shipment crosses a risk threshold, with ranked reroute/hold solutions.

- Manager dashboard for reviewing and resolving disruptions.
- Field operator dashboard for status updates and receiving resolution notifications.
- Admin panel for user, hub, and cargo-type management, plus an audit log.
- Shipment management (create, edit, delete) for managers and admins, with server-side validation.
- Reports with date-range filters and CSV export (disruption summary, resolution time, shipment risk, audit log).
- An AI assistant (Gemini, with an offline fallback) for natural-language questions about shipment data.
- Live map visualisation of shipments and disruption zones.
- A demo/simulation mode — simulated shipment movement, a one-click “Trigger demo disruption” action and a reset command — for showing the system without live production data.

Outside the scope of this academic build:

- Integration with live carrier/GPS hardware feeds (simulated positions are used instead).
- Payment, billing, or contractual/SLA enforcement modules.
- Multi-tenant support for multiple independent companies.

1.7 Advantages and Limitations of the Proposed System

Advantages:

- Shifts the organisation from reactive to preemptive disruption handling.
- Quantifies risk with a transparent, weighted scoring formula instead of gut feeling.
- Cuts the time between “risk detected” and “operator notified” to seconds.
- Role-based views reduce information overload for each type of user.
- Runs on a lightweight Flask + SQLite stack, containerised with Docker, so it is cheap to host and easy to demonstrate on any machine.

Limitations:

- Near-real-time, not instantaneous — performance depends on the scheduler interval (60 seconds by default).
- Weather and route-congestion signals rely on external APIs, or a simulated fallback, rather than certified logistics-grade data feeds.
- SQLite suits a single-node academic prototype but would need to be replaced by PostgreSQL/MySQL for real multi-user production load.
- The AI assistant's answers are only as reliable as the underlying Gemini model and the data exposed to it — it is a convenience feature, not a system of record.

1.8 Proposed Time Line Chart

The project was planned across the semester using the phased schedule below (Fig 1.1). Table 1.1 records the milestones actually achieved, taken from the project's version-control history and document records.

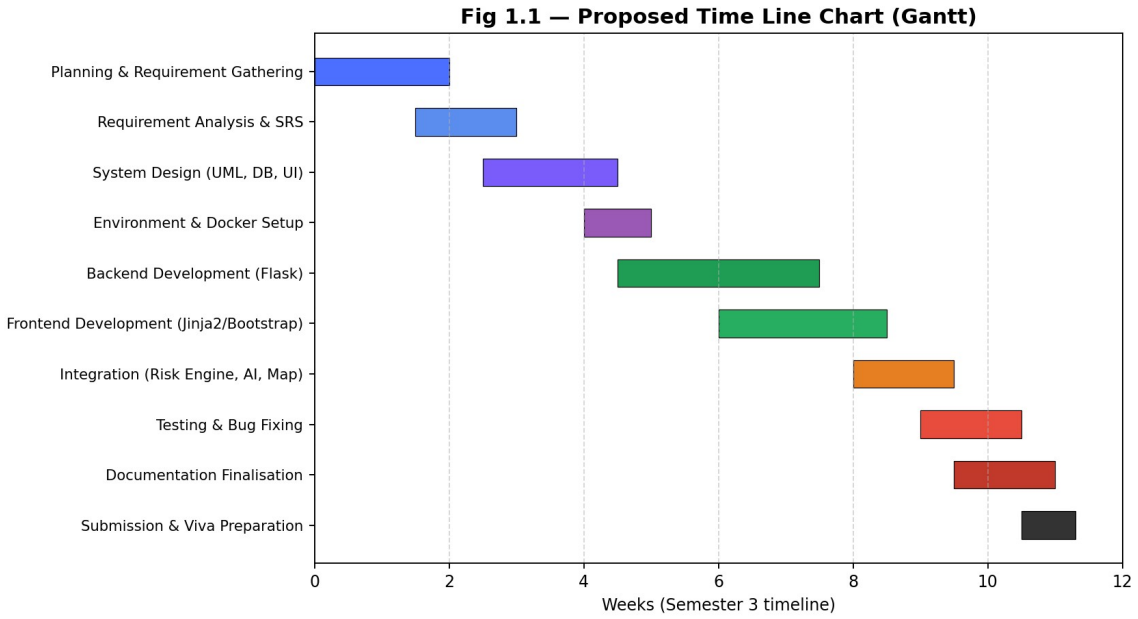


Fig 1.1 — Proposed Time Line Chart

Table 1.1 — Actual Progress Against the Plan

Milestone	Completed On	Status
Problem analysis and technology stack decision	13 Aug 2026	Done
Project report — requirements and system design	15–19 Aug 2026	Done
Development environment and Docker setup	16 Aug 2026	Done
Authentication and base user interface	16–25 Aug 2026	Done
SRS document and first presentation	25 Aug 2026	Done
Risk engine scan loop, route solver, disruption module, admin panel	7–8 Sep 2026	Done
Shipment CRUD, live map, dashboards and charts, real-time notifications, chat, AI assistant	24 Sep 2026	Done
Reports and CSV export, cargo/hub/user management, demo mode	24 Sep 2026	Done
Testing (101 automated checks) and Docker deployment verification	24 Sep 2026	Done
Documentation update (v2)	24 Sep 2026	Done
Final submission and viva	As per university schedule	Pending

1.9 Targeted User

- Logistics / Supply Chain Manager — the primary decision-maker who reviews and resolves disruptions.
- Field Operators — Captain, Pilot, Driver, and Loco Pilot — who execute the resolved route.
- System Administrator — manages users, hubs, cargo types, and system configuration.
- Indirectly, end customers and downstream partners who benefit from fewer missed deliveries.

2. REQUIREMENT DETERMINATION AND ANALYSIS

2.1 Requirement Determination

Requirements for ChainShield Transportation System were determined through a combination of group discussion, structured analysis of the Smart Supply Chains problem statement, and review of the team's earlier ChainShield prototype (built on FastAPI/React for a solution-building initiative) as a feature reference point.

The group broke the problem down into what a supply-chain manager needs to see, what a field operator needs to act on, and what an administrator needs to configure — and cross-checked each requirement against the objective of catching disruptions before they cascade, rather than after.

Key requirements identified through this process:

- All users (admin, manager, field operator) must authenticate before accessing the system.
- The system must continuously evaluate risk for every active shipment without manual triggering.
- A disruption must be created automatically once a shipment's risk score reaches MEDIUM (≥ 50), and escalated if it rises to HIGH (≥ 70).
- The system must propose at least three ranked route-solution alternatives per disruption (hold, reroute via a hub, mode switch, or continue).
- Field operators must be notified in-app the moment a manager resolves a disruption in their favour.
- Admins must be able to manage users, hubs, and cargo types, and view an audit trail of system activity.
- The system must be demonstrable without live production data, via a simulated/demo disruption trigger.

2.2 Requirement Specification

Functional Requirements:

- User registration, login, and role-based access control (Admin, Manager, Field Operator).
- Shipment create, read, update and delete (managers and admins), with field operators seeing only their assigned shipments.
- Continuous, scheduled risk scoring of all active shipments.
- Automatic disruption detection and alerting once a risk threshold is crossed.
- Generation of ranked route-solution alternatives (hold, hub reroute, mode switch, continue) with cost, time, and residual-risk trade-offs.
- Real-time in-app notification delivery (Socket.IO) to managers and the assigned field operator on detection, escalation and resolution of a disruption.
- Direct messaging (chat) between users.
- Admin CRUD operations for users, logistics hubs, and cargo types.

- Activity audit log for all administrative and disruption-resolution actions.
- AI assistant for natural-language queries about the supply chain.
- Reports with date-range filter and CSV export.
- Live map rendering of shipment positions and disruption zones.
- A one-click demo-mode trigger for presentation purposes.

Non-Functional Requirements:

- Performance — risk scores must refresh at least once every 60 seconds.
- Usability — each role must see a dashboard tailored to its responsibilities, without unnecessary clutter.
- Portability — the system must run identically on any machine via Docker, without manual environment setup.
- Reliability — database writes must be transactional; the background scheduler must recover gracefully from a failed run.
- Security — passwords must be hashed (never stored in plain text); sessions managed via Flask-Login; all forms CSRF-protected; admin rights cannot be self-assigned at registration.
- Maintainability — code must be organised into modular Flask blueprints so features can be extended independently.

Platform / Hardware & Software Requirements:

Operating System	Windows 11 / any OS supporting Docker
Backend Language	Python 3.10
Web Framework	Flask, Flask-SQLAlchemy, Flask-Login, Flask-SocketIO, Flask-WTF, APScheduler
Database	SQLite
Containerisation	Docker, Docker Compose (Gunicorn + eventlet worker)
Front-End Tools	HTML5, CSS3, Bootstrap 5, Jinja2, JavaScript, Leaflet.js, Chart.js
External APIs	Google Gemini API, OpenWeatherMap API
Documentation Tool	Microsoft Word 2021

3. SYSTEM DESIGN

3.1 Use Case Diagram

The diagram below summarises the primary use cases for each of the three system roles — Admin, Manager, and Field Operator — plus the background scheduler that scores risk automatically, within the ChainShield Transportation System boundary.

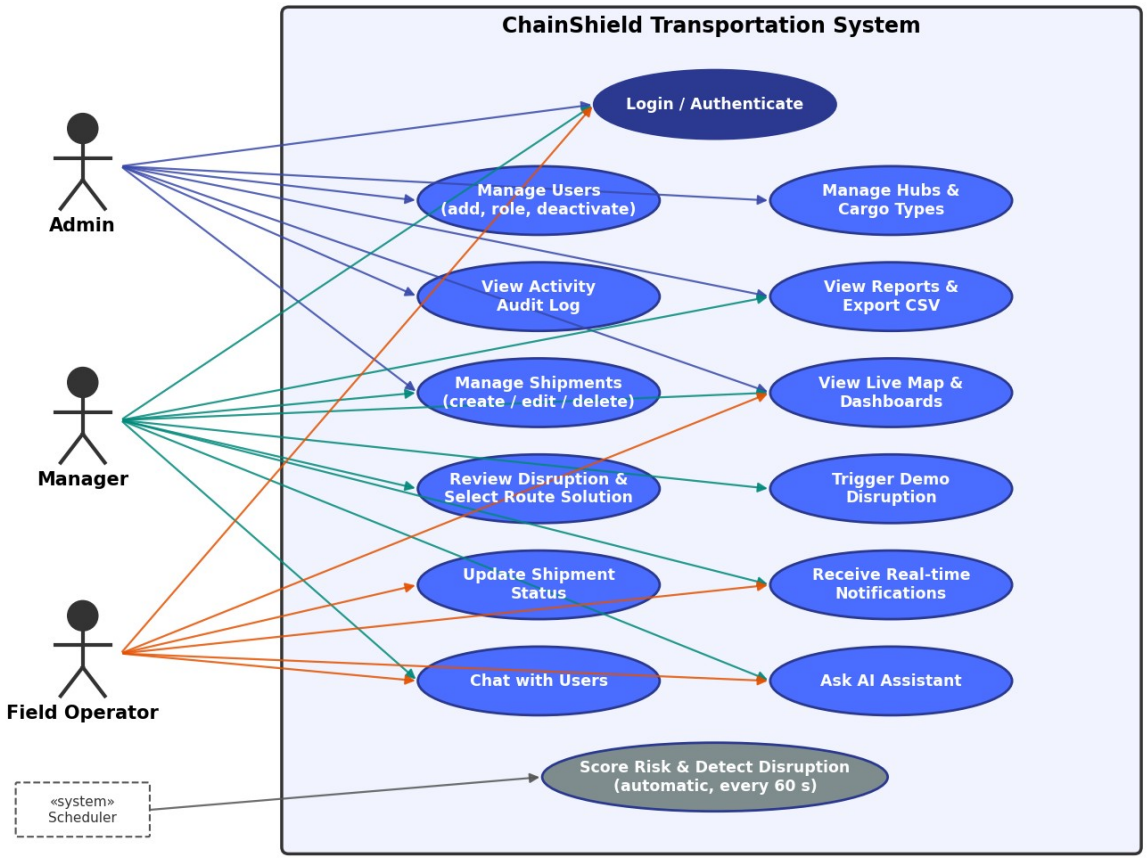


Fig 3.1 — Use Case Diagram

System Architecture:

The system follows a layered architecture: a Flask + Jinja2 + Bootstrap presentation layer (with Leaflet.js and Chart.js) talking to the server over HTTP and WebSockets, core services (Risk Engine, Route Solver, Notifications/Chat, AI Assistant, Admin & Reports) in the middle, a Flask-SQLAlchemy data access layer, and a SQLite database at the base — the entire stack packaged inside a Docker container.

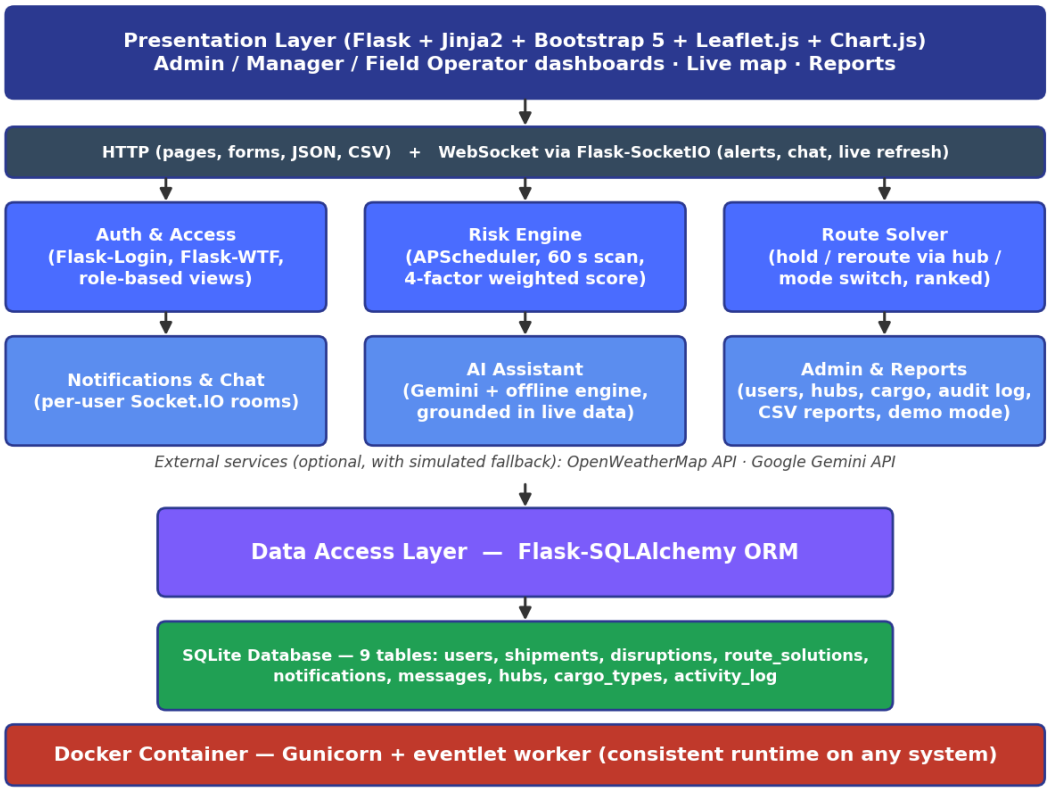


Fig 3.2 — ChainShield System Architecture

3.2 Data Dictionary

Database: chainshield

1. users

Column Name	Data Type	Description
id	INTEGER (PK)	Unique user identifier
name	VARCHAR(100)	Full name
email	VARCHAR(100)	Login email (unique)
password_hash	VARCHAR(255)	Hashed password (Werkzeug)
role	VARCHAR(20)	admin / manager / captain / pilot / driver / loco_pilot
phone	VARCHAR(20)	Contact number
is_active	BOOLEAN	False = account deactivated by an admin (cannot log in)
created_at	DATETIME	Account creation timestamp

2. shipments

Column Name	Data Type	Description
id	INTEGER (PK)	Unique shipment identifier
tracking_no	VARCHAR(30)	Human-readable tracking number (auto CS-xxxx)
origin / destination	VARCHAR(100)	Origin and destination hub cities
cargo_type_id	INTEGER (FK)	References cargo_types.id
transport_mode	VARCHAR(20)	Road / Rail / Sea / Air
status	VARCHAR(20)	In-transit / Delayed / On-hold / Delivered
current_lat / current_lng	FLOAT	Live (simulated) position
risk_score	FLOAT	Composite risk score (0-100)
risk_level	VARCHAR(15)	LOW / AT-RISK / MEDIUM / HIGH
cargo_value	FLOAT	Declared cargo value (INR)
assigned_operator_id	INTEGER (FK)	References users.id
hub_id	INTEGER (FK)	References hubs.id (origin hub)
created_at	DATETIME	Shipment creation timestamp

3. disruptions

Column Name	Data Type	Description
id	INTEGER (PK)	Unique disruption identifier
shipment_id	INTEGER (FK)	References shipments.id
type	VARCHAR(30)	Dominant factor: Weather / Congestion / Cargo / History
severity	VARCHAR(15)	MEDIUM / HIGH (escalated as risk rises)
risk_score	FLOAT	Score at detection (updated on escalation)
status	VARCHAR(15)	Open / Resolved
detected_at / resolved_at	DATETIME	Lifecycle timestamps (used for resolution time)
resolved_by	INTEGER (FK)	References users.id (manager/admin)

4. route_solutions

Column Name	Data Type	Description
id	INTEGER (PK)	Unique solution identifier
disruption_id	INTEGER (FK)	References disruptions.id

option_no	INTEGER	Rank, 1 = recommended (up to 5 options)
description	TEXT	e.g. "Reroute via Jaipur Hub (+77 km detour)"
extra_time_hours	FLOAT	Additional transit time
extra_cost	FLOAT	Additional cost (INR)
risk_score	FLOAT	Residual risk if chosen
selected	BOOLEAN	Whether the manager chose this option

5. notifications, messages, hubs, cargo_types, activity_log

Column Name	Data Type	Description
notifications.id / user_id / message / is_read / created_at	mixed	In-app alerts delivered to a specific user
messages.id / sender_id / receiver_id / chat_id / content / sent_at	mixed	Direct chat messages between users
hubs.id / name / location / lat / lng	mixed	Logistics hub master data
cargo_types.id / name / sensitivity_weight	mixed	Cargo master data used in risk scoring
activity_log.id / user_id / action / timestamp	mixed	Audit trail of administrative actions

3.3 User Interface Design

The interface is built with server-rendered Jinja2 templates and Bootstrap 5, sharing a common base layout (navigation bar with a live notification bell). The implemented screens are listed below; screenshots of each are given in Section 4.3.

- Login / Register — authentication; admin rights can only be granted by an existing admin.
- Admin Dashboard — KPI cards (users, shipments, disruptions, cargo value at risk), four live Chart.js charts and recent activity.
- Operations Dashboard — for managers and field operators: KPIs, charts, a risk watchlist and open disruptions for the shipments they can see.
- Live Map — Leaflet map of shipments coloured by risk, hubs, planned routes and shaded disruption zones; refreshes after every scan.
- Disruptions — each open disruption with its ranked route options (time, cost, residual risk) and a one-click “Apply selected option”.
- Shipments — searchable list, detail page with mini-map and disruption history, and a validated create/edit form.
- Admin Panel — tabs for Users, Shipments, Base Cities (hubs), Cargo Types, Reports, System Config and Activity Log.

- Chat, Notifications and AI Assistant — real-time direct messages, a notification feed, and a chat-style assistant with suggested questions.

All screens follow a consistent colour system (green LOW, yellow AT-RISK, orange MEDIUM, red HIGH) and are responsive across desktop, tablet and mobile breakpoints via Bootstrap's grid system.

3.4 Report Design

The Reports module (available to admins and managers) shows the following reports on-screen, filtered by a selected date range; the reports marked CSV can be exported and opened directly in Excel:

- Disruption Summary Report (CSV) — count of disruptions by dominant factor and severity, with each disruption's detection time, resolution time, decision-maker and chosen option.
- Shipment Risk Report (CSV) — current risk score, level, status, cargo value and operator for every shipment.
- Resolution Time Report — average time from disruption detection to a route decision, per manager/admin.
- Activity Audit Log (CSV) — a chronological record of logins, administrative actions, shipment changes and disruption resolutions, for accountability.

4. DEVELOPMENT

4.1 Coding Standards

The following coding standards were followed throughout development, keeping the codebase consistent, readable, and easy for all three group members to work on in parallel:

- PEP 8 style guide for all Python code, with 4-space indentation.
- Modular Flask Blueprint structure — separate blueprints for auth, main (dashboards, admin, reports), shipments, disruptions, notifications, chat, and the AI assistant.
- Business logic kept out of route handlers — risk scoring, route solving, weather, notifications, statistics and the assistant live in a dedicated services/ package.
- Template inheritance via a single base.html in Jinja2, so shared layout (navbar, sidebar, footer) is defined once.
- Meaningful, descriptive names for variables, functions, and database columns — no single-letter names outside loop counters.
- Secrets and configuration (API keys, database URL) stored in environment variables via a .env file — never hard-coded.
- Docstrings and inline comments for non-trivial logic, particularly the risk-scoring formula and route-solution ranking.
- Git version control with descriptive commit messages, and an automated test script run before each commit.
- Dependency versions pinned in requirements.txt for reproducible builds inside Docker.

4.2 Implementation Details

Risk Engine (app/services/risk_engine.py)

- APScheduler runs a scan every 60 seconds (configurable). Each in-transit shipment is scored: score = 0.40 × weather + 0.25 × congestion + 0.20 × cargo sensitivity + 0.15 × lane history (each 0–100).
- Thresholds: ≥ 70 HIGH, ≥ 50 MEDIUM, ≥ 30 AT-RISK, otherwise LOW. AT-RISK shipments are watched; a disruption is opened at MEDIUM and escalated (with re-planned options) if the shipment reaches HIGH.
- Weather comes from OpenWeatherMap when a key is configured; otherwise a simulated signal is used (mostly calm, with an occasional storm). Lane history grows with disruptions on the same origin–destination lane in the last 90 days.
- In demo mode each scan also moves shipments towards their destination hub, marking them Delivered on arrival.

Route Solver (app/services/route_solver.py)

- Candidates: hold at the nearest hub, continue (baseline), reroute via the two hubs with the smallest detour (haversine distance), and a mode switch for the remaining leg (e.g. Road → Rail).

- Each option gets extra time, extra cost (₹, from per-km mode rates) and residual risk, and is ranked by 0.60 × risk + 0.25 × normalised time + 0.15 × normalised cost — option 1 is the recommendation.

Real-time Layer (Flask-SocketIO)

- Every logged-in browser joins a personal room (user_<id>). Notifications and chat messages are stored in the database and pushed to the right room; broadcast events (shipments_updated, disruption_new) make the live map, dashboards and charts refresh themselves without polling.

AI Assistant (app/services/assistant.py)

- The question and a snapshot of live data the user is allowed to see (shipments, risk, open disruptions and their ranked options) are sent to Google Gemini. Without a key or internet, a rule-based offline engine answers from the same data, so the feature always works in a demonstration.

Security and Deployment

- Werkzeug password hashing, Flask-Login sessions, CSRF protection on every form and AJAX call, role checks on every route, account deactivation, and no self-registration as admin.
- Docker image on Python 3.10 served by Gunicorn with an eventlet worker (real WebSockets); the SQLite file is persisted through a volume and secrets are supplied through a .env file that is never committed.

4.3 Output Screens

The screenshots below were captured from the running application with demo data (the live-map captures use a simplified country-outline basemap; in normal use the map shows OpenStreetMap tiles).

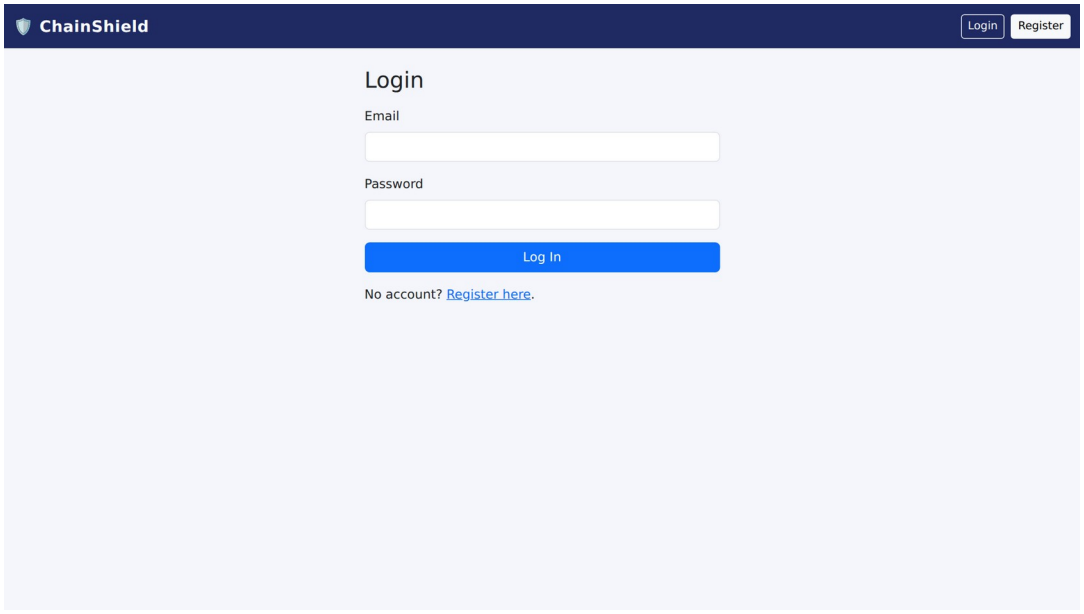


Fig 4.1 — Login Screen

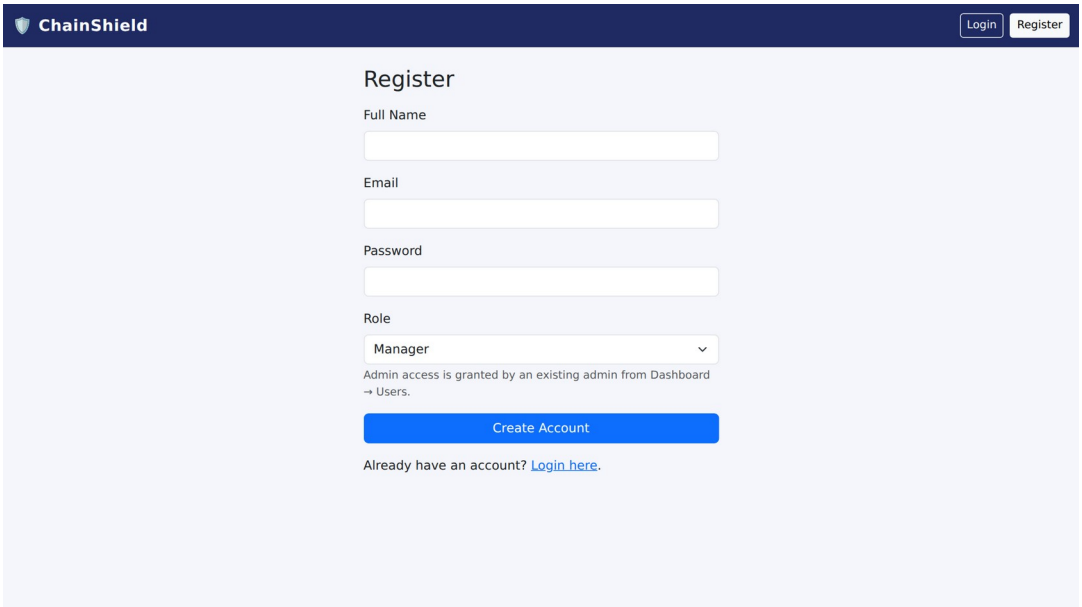


Fig 4.2 — Registration Screen

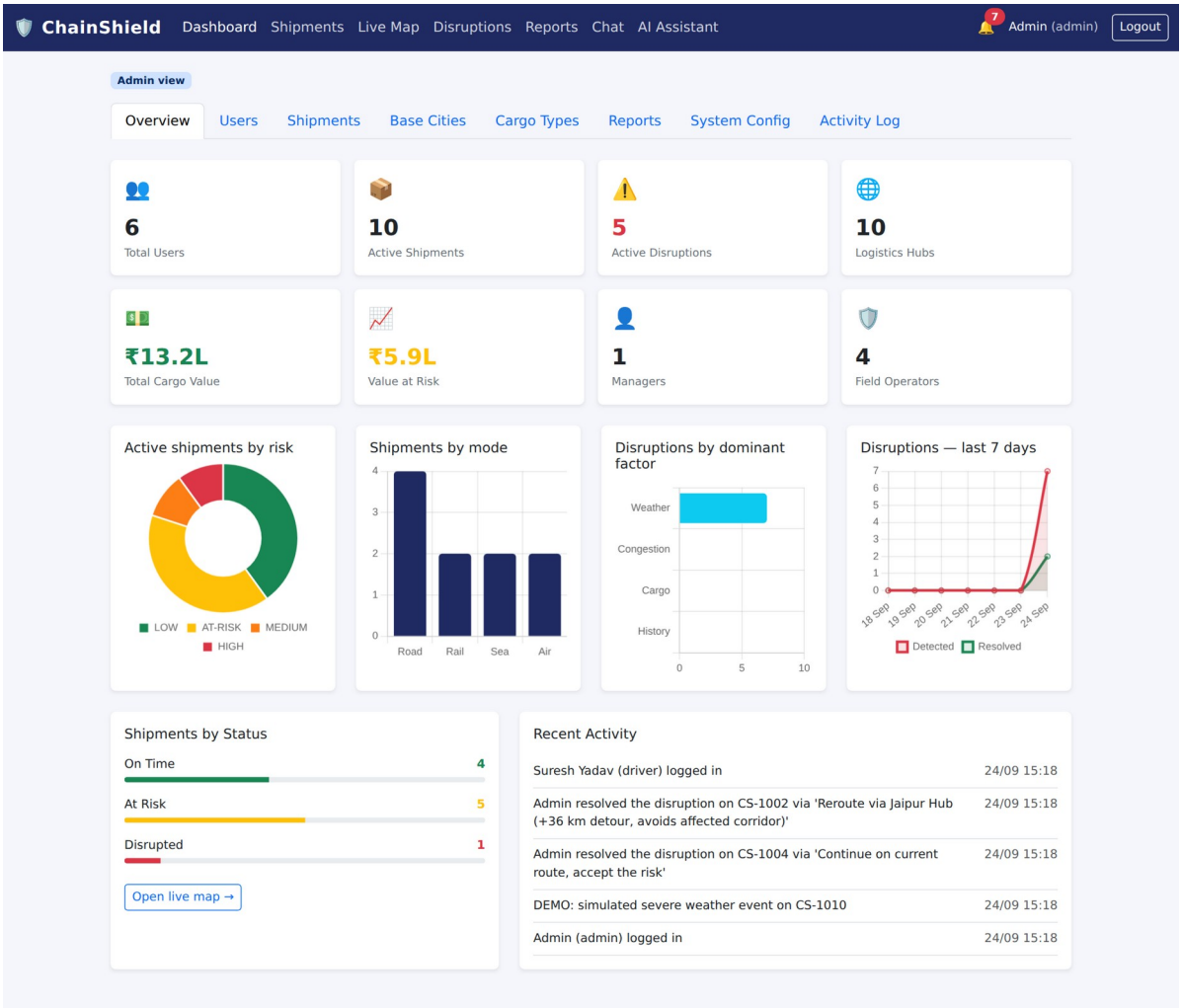


Fig 4.3 — Admin Dashboard with Live Charts

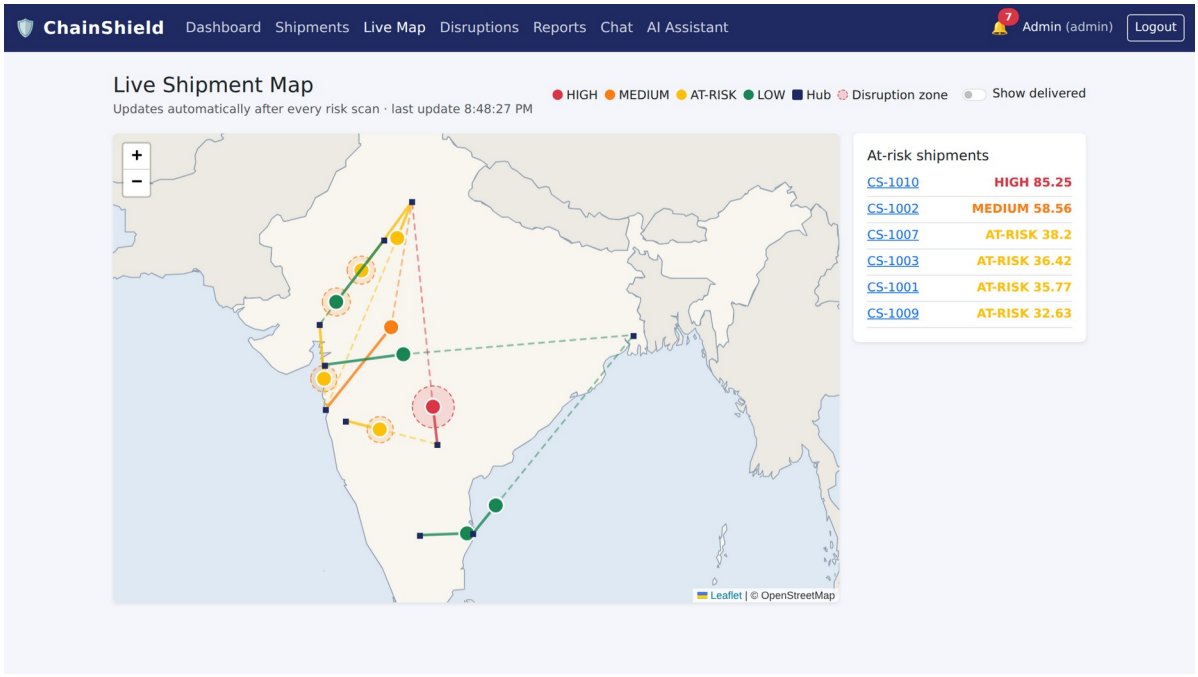


Fig 4.4 — Live Shipment Map with Disruption Zones

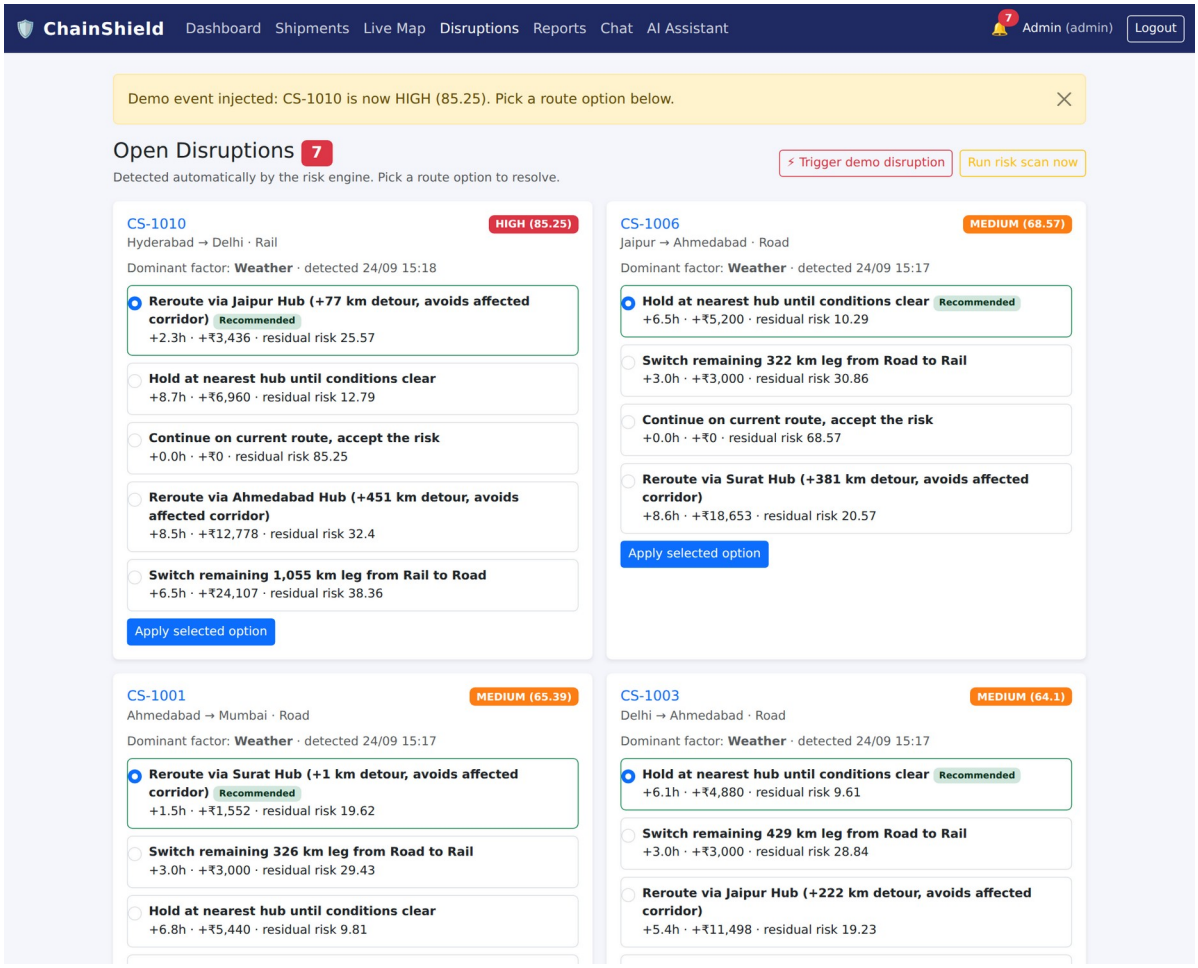


Fig 4.5 — Disruptions with Ranked Route Options

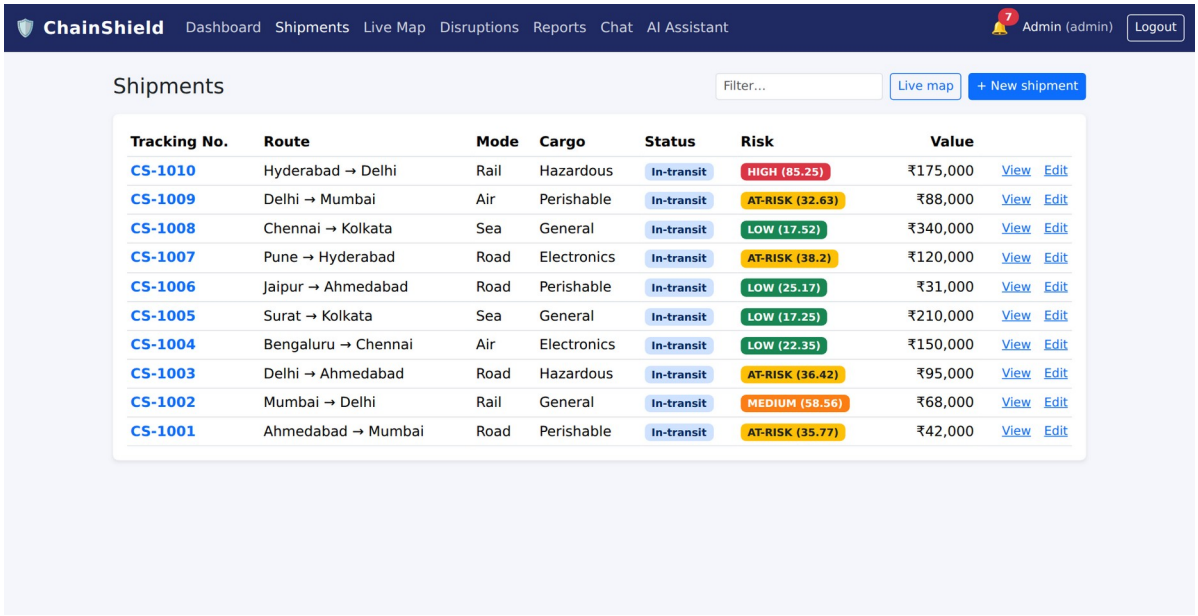


Fig 4.6 — Shipment List

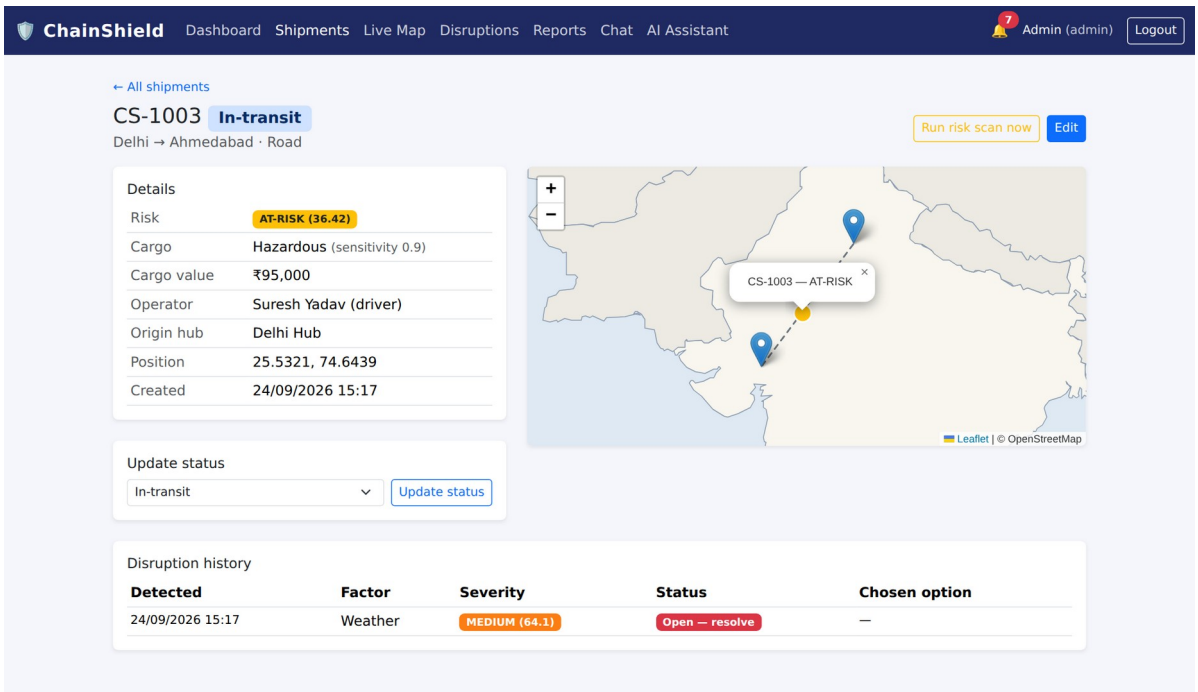


Fig 4.7 — Shipment Detail with Mini-map and History

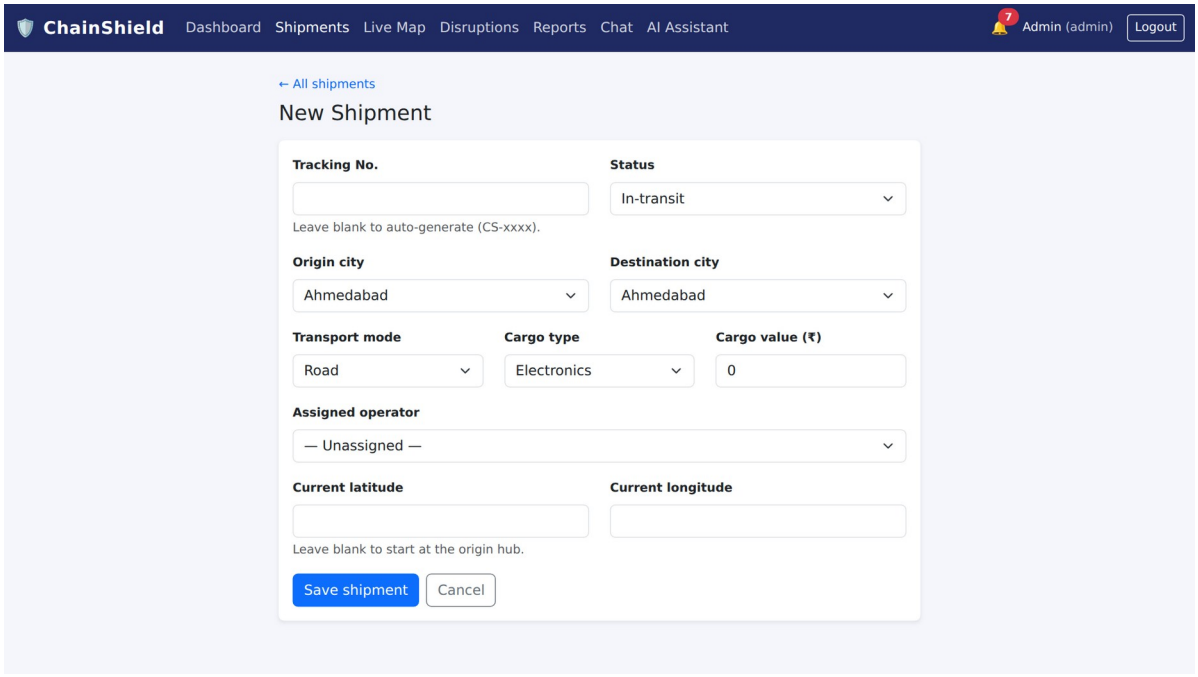


Fig 4.8 — Create Shipment Form

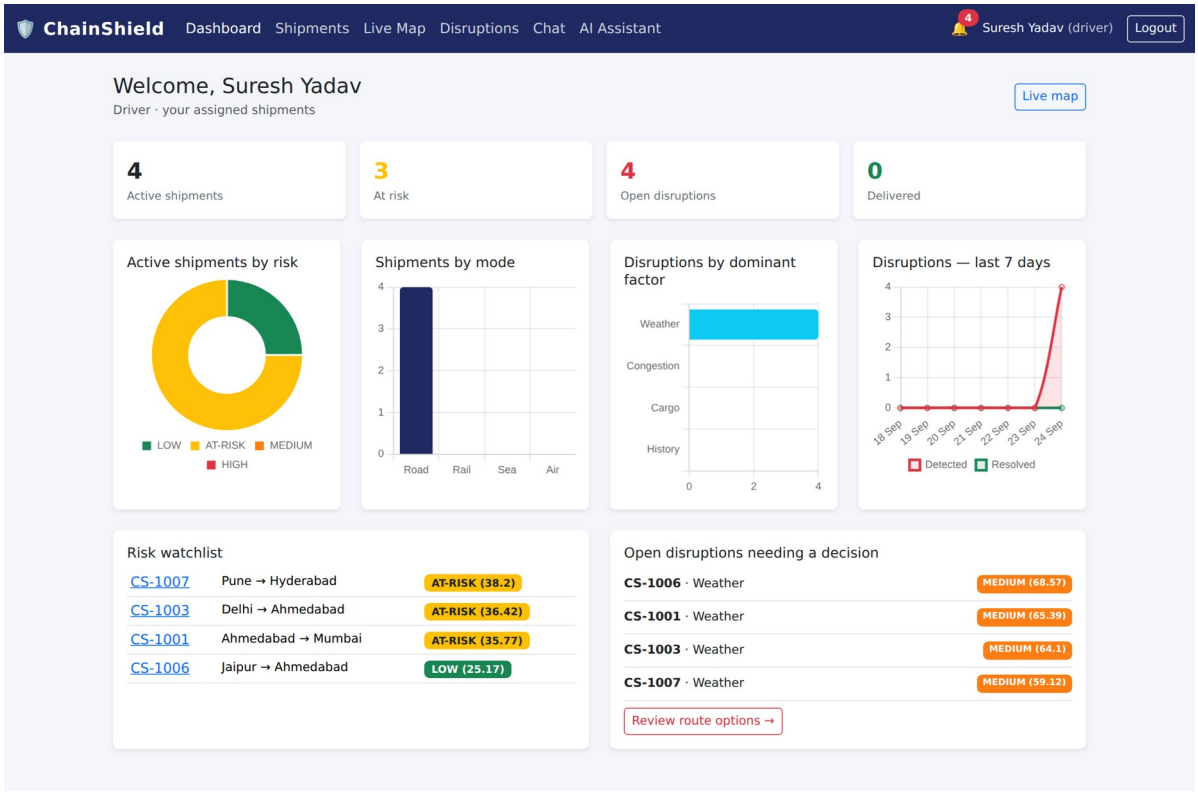


Fig 4.9 — Field Operator (Driver) Dashboard

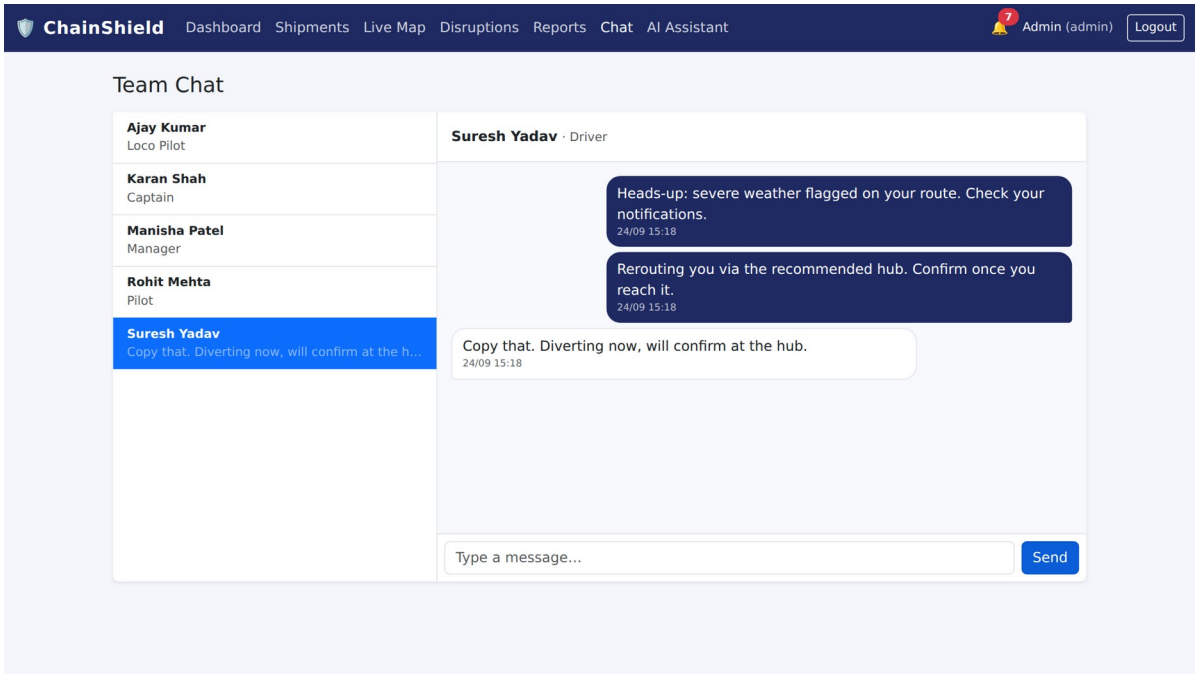


Fig 4.10 — Real-time Team Chat

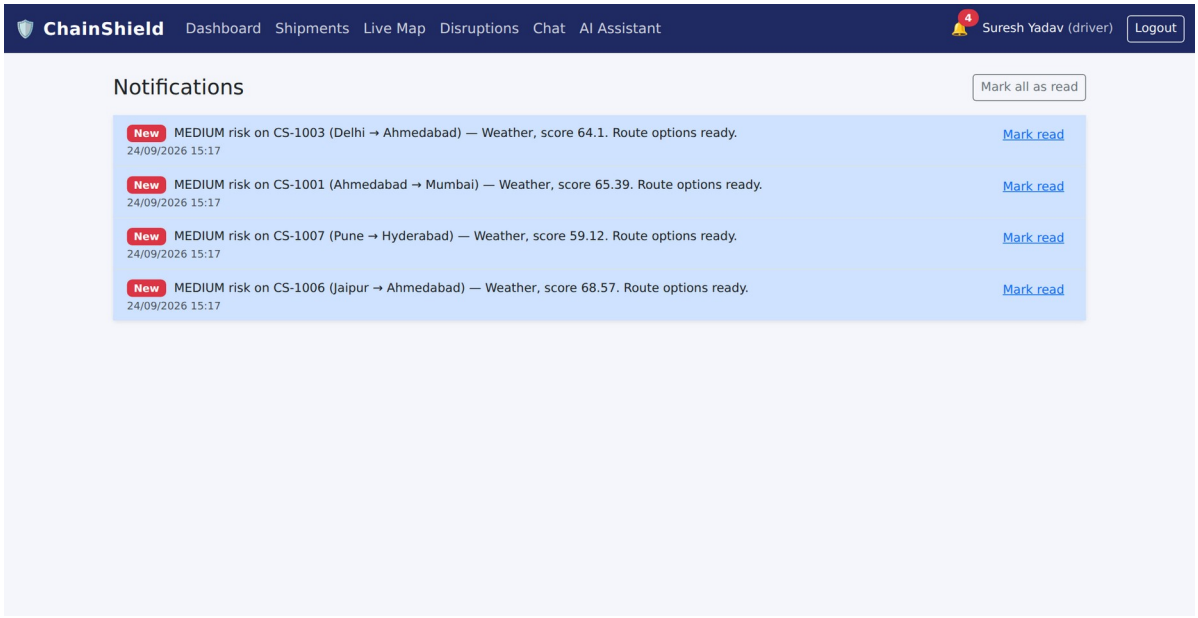


Fig 4.11 — Notification Feed

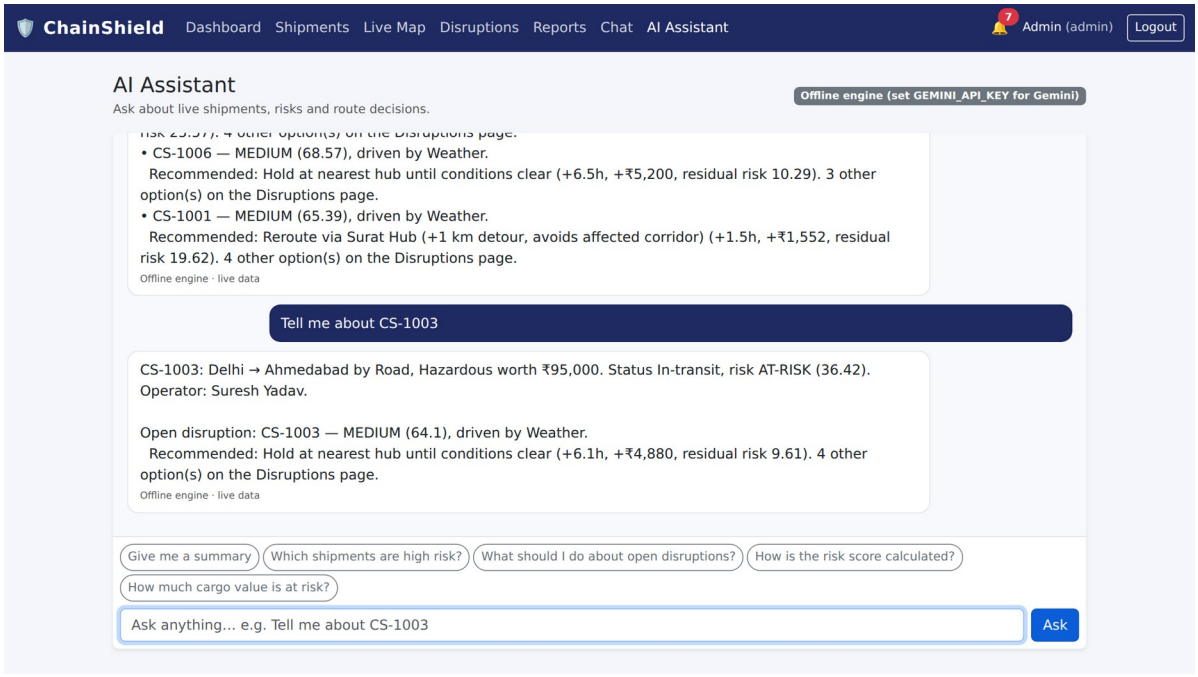


Fig 4.12 — AI Assistant

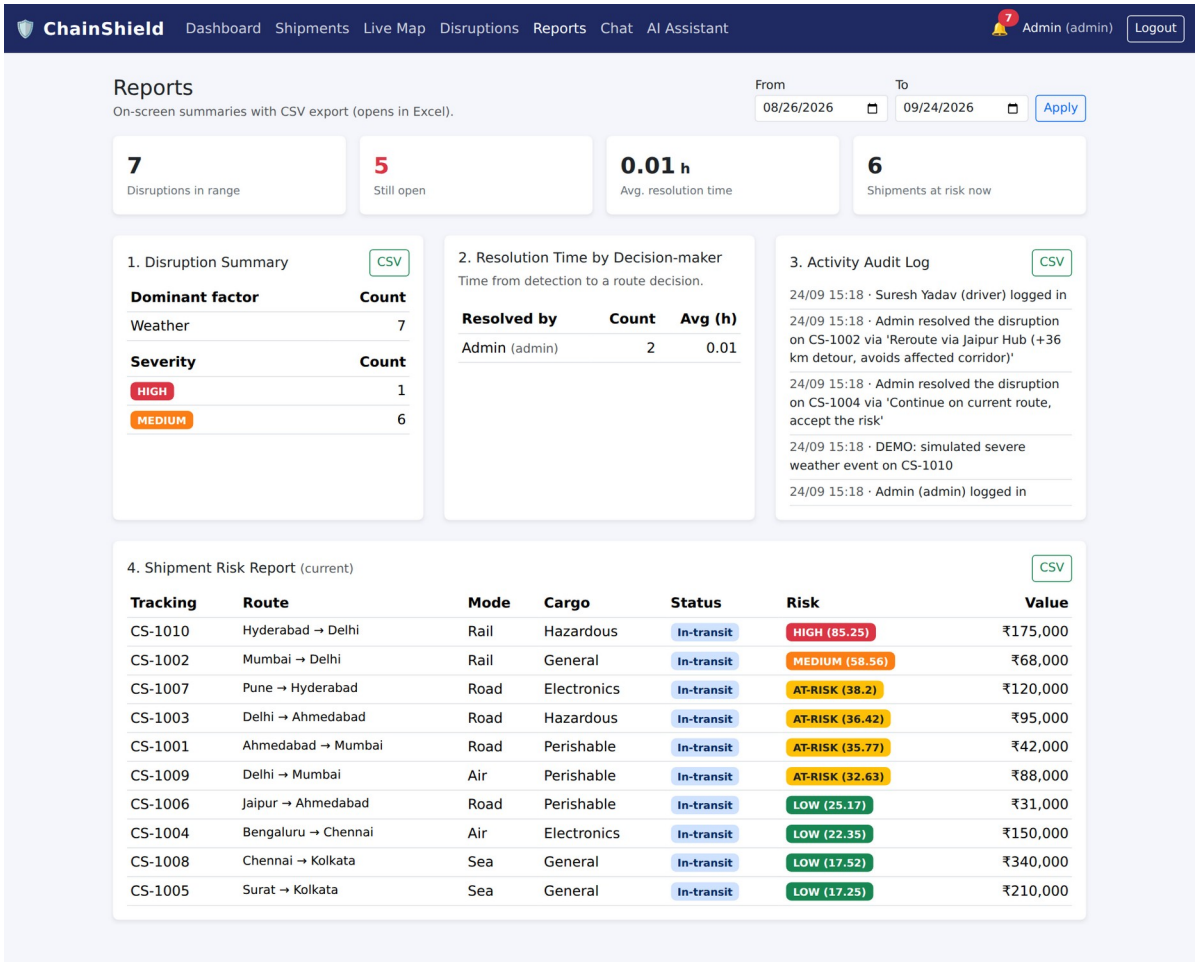


Fig 4.13 — Reports with CSV Export

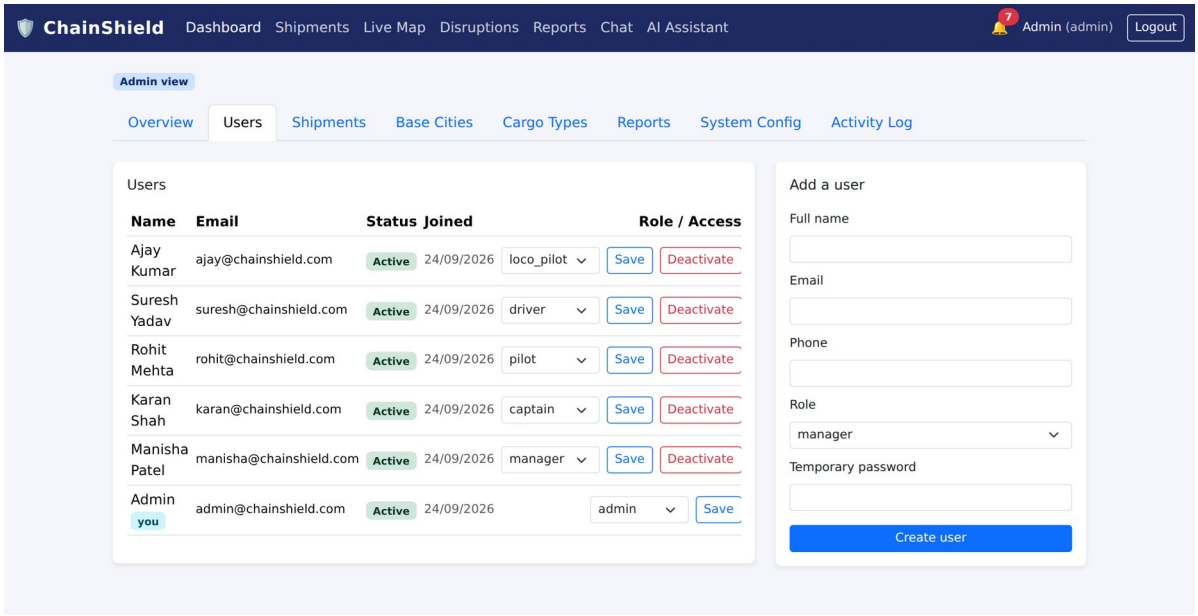


Fig 4.14 — Admin — User Management

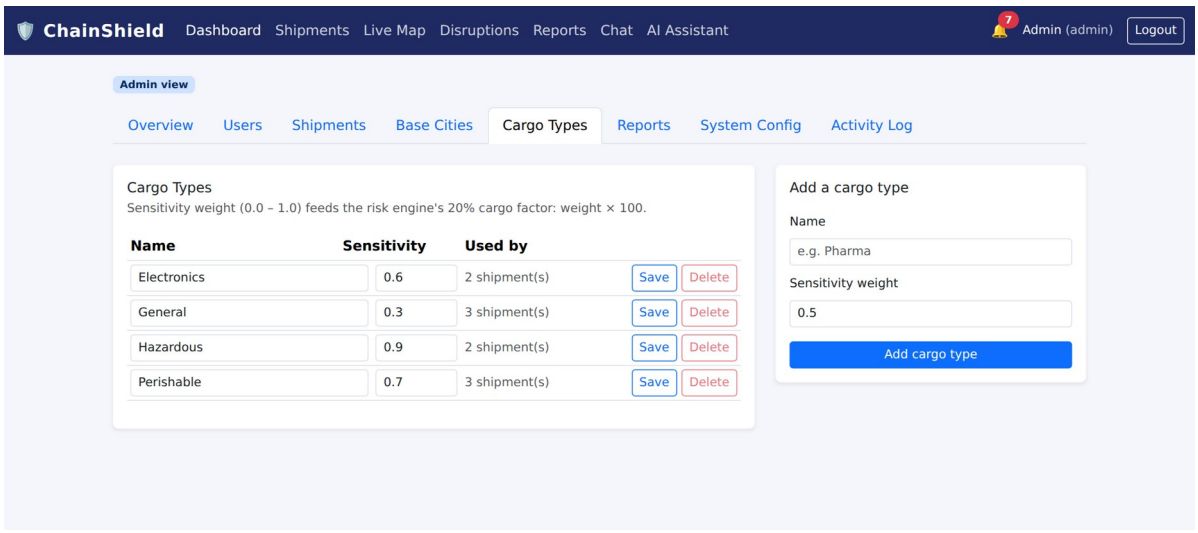


Fig 4.15 — Admin — Cargo Types

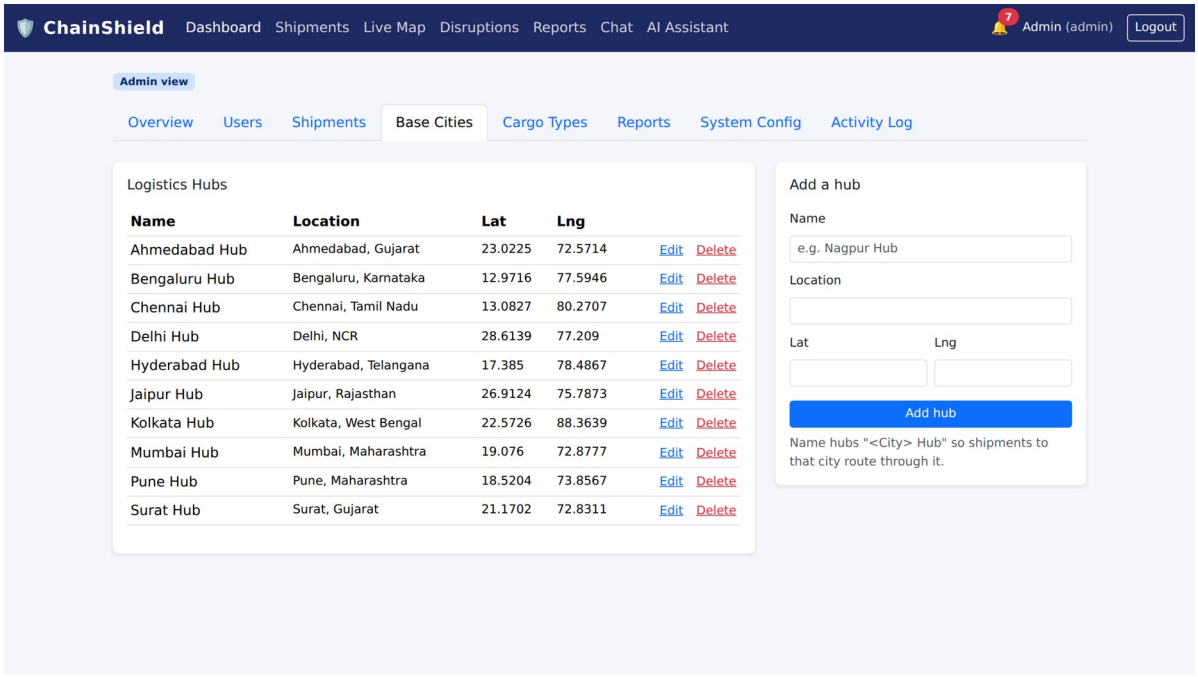


Fig 4.16 — Admin — Logistics Hubs

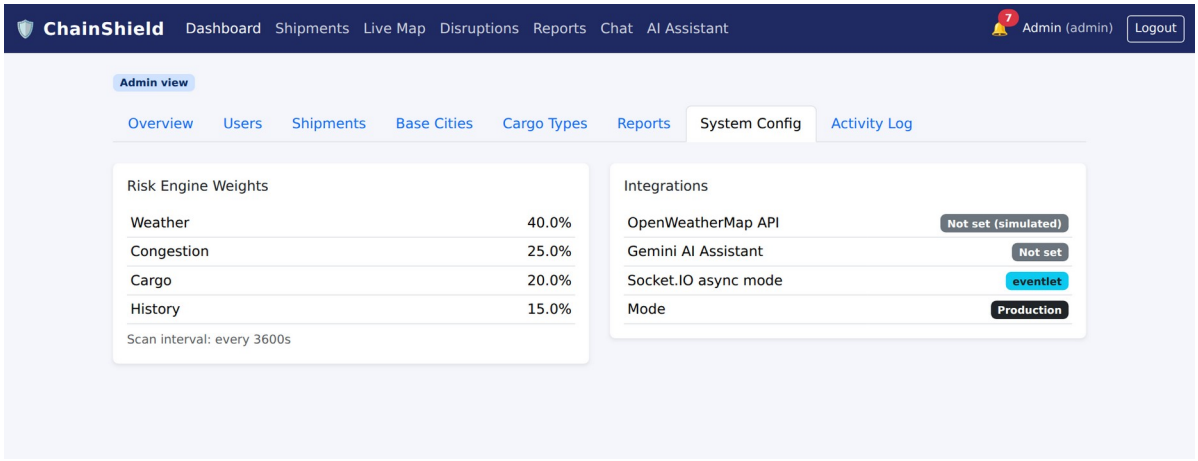


Fig 4.17 — Admin — System Configuration

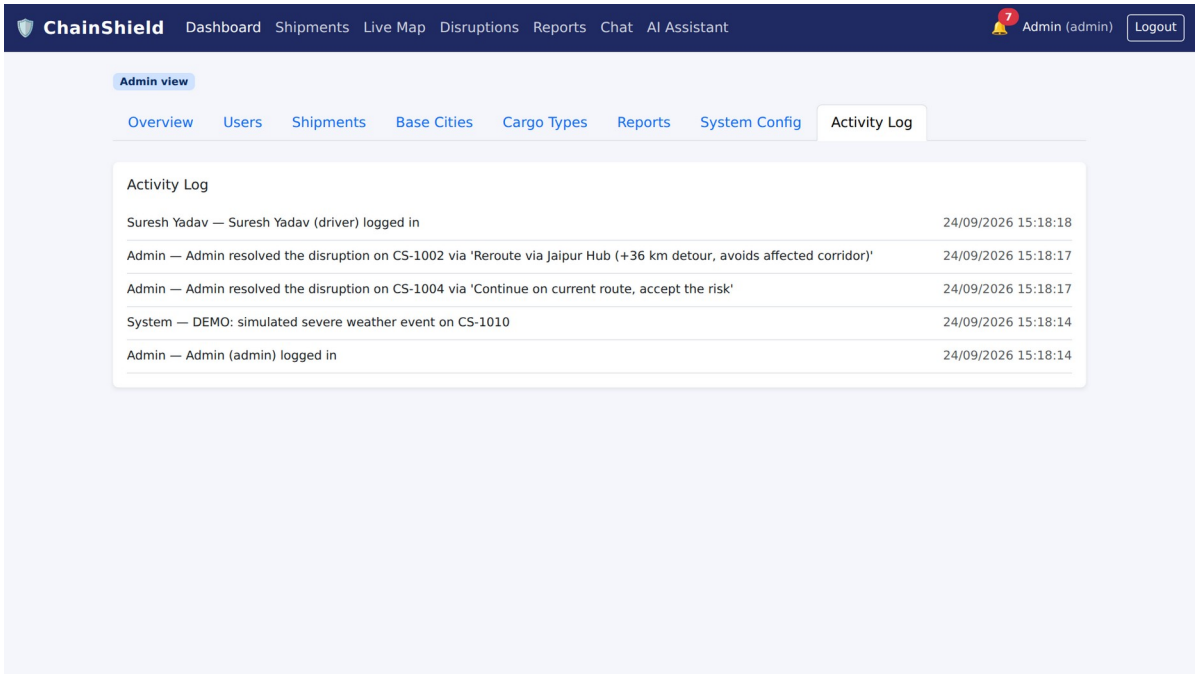


Fig 4.18 — Admin — Activity Audit Log

5. TESTING

5.1 Test Cases

Testing combined an automated test script (101 checks run through Flask's test client and the Socket.IO test client, covering every route, role restriction, the risk and route-solver logic, real-time events, chat, the assistant, reports and admin CRUD), manual testing of each screen in the browser, and a Docker deployment test. Representative test cases and their results are listed below.

Test Case 1 — User Login

Test ID	TC001
Description	Verify that a registered user can log in with correct credentials.
Pre-conditions	User account exists in the database.
Test Data	email = manisha@chainshield.com, password = demo1234
Expected Result	User is authenticated and redirected to their role-specific dashboard.
Status	Pass

Test Case 2 — Risk Score Calculation

Test ID	TC002
Description	Verify the risk engine correctly computes a composite score from the 4 weighted factors.
Pre-conditions	Risk weights configured as 0.40 / 0.25 / 0.20 / 0.15.
Test Data	Weather=80, Congestion=60, Cargo=50, History=40
Expected Result	Composite score = 0.40×80 + 0.25×60 + 0.20×50 + 0.15×40 = 63 (MEDIUM)
Status	Pass

Test Case 3 — Automatic Disruption Creation and Escalation

Test ID	TC003
Description	Verify a disruption is auto-created when a shipment's score reaches MEDIUM (≥ 50) and escalated when it reaches HIGH (≥ 70).
Pre-conditions	Scheduler running; an in-transit shipment with no open disruption.
Test Data	Scan 1: weather = congestion = 70 (MEDIUM); Scan 2: weather = congestion = 100 (HIGH)
Expected Result	One disruption is opened at MEDIUM with ≥ 3 ranked route solutions and alerts are sent; on scan 2 the same disruption is escalated to HIGH, its options re-planned and an “ESCALATED” notification sent.
Status	Pass

Test Case 4 — Route Solution Selection & Notification

Test ID	TC004
Description	Verify that resolving a disruption notifies the assigned field operator instantly.
Pre-conditions	An open disruption with ranked route solutions exists; manager is logged in.
Test Data	Manager selects Option 1 (recommended), e.g. “Reroute via Jaipur Hub”
Expected Result	Disruption status becomes Resolved; the operator receives a live notification and toast; the action appears in the audit log.
Status	Pass

Test Case 5 — Admin CRUD (Cargo Type)

Test ID	TC005
Description	Verify admin can add, edit, and delete a cargo type.
Pre-conditions	Admin is logged in.
Test Data	name = 'Pharma', sensitivity_weight = 0.9 → edited to 0.85; weight 1.5 attempted
Expected Result	Cargo type is saved, edited and deleted; weight outside 0.0–1.0 is rejected; deleting a cargo type used by shipments is blocked.
Status	Pass

Test Case 6 — Docker Container Startup

Test ID	TC006
Description	Verify the application starts correctly inside a Docker container with no manual configuration.
Pre-conditions	Docker Desktop installed on Windows 11.
Test Data	docker compose up --build
Expected Result	Image builds; Gunicorn starts with the eventlet worker on port 5000; the app, database and scheduler run and the site is reachable at http://localhost:5000.
Status	Pass

Test Case 7 — Role-Based Access Control

Test ID	TC007
Description	Verify field operators cannot reach admin/manager functions and only see their own shipments.
Pre-conditions	Driver account with 3 assigned Road shipments.
Test Data	GET /dashboard/users, /shipments/new, /reports; view another operator's shipment
Expected Result	403 Forbidden for admin/manager pages; 404 for other operators' shipments; live map lists only the driver's 3 shipments.
Status	Pass

Test Case 8 — Real-time Chat Delivery

Test ID	TC008
Description	Verify a chat message is stored and delivered live to the recipient.
Pre-conditions	Admin and driver both logged in (two browsers).
Test Data	Admin sends “Hold CS-1001 at Surat” to the driver
Expected Result	Message saved; driver's open socket receives it instantly; it appears in both chat histories; empty messages are rejected.
Status	Pass

Test Case 9 — AI Assistant with Fallback

Test ID	TC009
Description	Verify the assistant answers from live data, and still answers when Gemini is unavailable.
Pre-conditions	Shipments and open disruptions exist.
Test Data	“Give me a summary”, “Tell me about CS-1003”, with a valid key and with an invalid key
Expected Result	Gemini answer with a valid key; with an invalid key the offline engine answers from the same data within seconds.
Status	Pass

Test Case 10 — Reports and CSV Export

Test ID	TC010
Description	Verify reports render and export correctly for managers, with a working date filter.
Pre-conditions	Manager logged in; disruptions and activity exist.
Test Data	/reports, /reports/disruptions.csv, ?from=2020-01-01&to=2020-01-02
Expected Result	Page shows 4 reports; CSV downloads open in Excel with resolution time column; out-of-range filter returns only the header row; drivers get 403.
Status	Pass

Test Case 11 — User Deactivation

Test ID	TC011
Description	Verify a deactivated user can no longer log in and an open session is ended.
Pre-conditions	Admin logged in; target user has an active session.
Test Data	Admin clicks Deactivate on the user
Expected Result	Login is refused with a message; the user's open session is logged out on the next request; admin cannot deactivate themself.
Status	Pass

Test Case 12 — Demo Disruption Trigger

Test ID	TC012
Description	Verify the demo trigger creates a live HIGH disruption instantly.
Pre-conditions	Manager logged in; at least one in-transit shipment.
Test Data	Click “Trigger demo disruption”
Expected Result	A HIGH disruption with ranked options appears immediately, a shaded zone is drawn on the live map, and alerts are pushed.
Status	Pass

6. PROPOSED ENHANCEMENT

The following enhancements are identified as valuable future work, beyond the scope of the current MCA Semester 3 submission:

- Integration with real GPS/telematics hardware feeds in place of simulated shipment positions.
- Migration from SQLite to PostgreSQL to support multi-node, production-scale deployment.
- A predictive machine-learning model, trained on historical disruption data, to augment or replace the current weighted-heuristic risk formula.
- A companion mobile application for field operators, for on-the-go status updates and notifications.
- Multi-tenant support so multiple independent logistics companies can use isolated instances of the platform.
- Additional notification channels — SMS and WhatsApp — alongside in-app alerts.
- Integration with public customs and port-authority APIs for verified, real-time congestion data.
- Improve the AI assistant — better prompts, conversation memory and suggested actions from its answers.
- PDF export of reports and scheduled e-mail delivery of the daily risk summary.

7. CONCLUSION

The ChainShield Transportation System addresses a well-defined, real-world problem: the tendency of supply-chain disruptions to be discovered only after delivery timelines are already compromised. By continuously scoring every active shipment against a transparent, weighted risk model, and by pairing every detected disruption with concrete, ranked route-solution alternatives, the system moves logistics management from a reactive posture to a preemptive one.

The project carried forward an initial prototype built for a solution-building initiative and rebuilt it from scratch for MCA Semester 3 on a simpler, self-contained Python (Flask) and SQLite stack, containerised with Docker so that it runs identically on any system. The original, already-deployed prototype was left untouched; the rebuild lives in its own separate project folder.

Group 59 translated the requirement analysis and system design in this report into a working, tested application: a scheduled risk engine, automatic disruption detection with escalation, a multi-objective route solver, a live map, role-based dashboards, real-time notifications and chat, a Gemini-powered assistant, reports with CSV export, and a full admin panel — verified by 101 automated checks and a Docker deployment test. It shows how a modest, open-source technology stack can meaningfully address a difficult operational problem in logistics.

8. BIBLIOGRAPHY

Books Referred

- Flask Web Development — Miguel Grinberg, O'Reilly Media
- Python Crash Course — Eric Matthes, No Starch Press
- Fundamentals of Database Systems — Ramez Elmasri & Shamkant B. Navathe, Pearson
- Software Engineering — Ian Sommerville, Pearson

Websites Referred

- https://flask.palletsprojects.com/
- https://docs.python.org/3/
- https://www.sqlite.org/docs.html
- https://docs.docker.com/
- https://getbootstrap.com/docs/5.3/
- https://leafletjs.com/reference.html
- https://developer.mozilla.org/
- https://ai.google.dev/gemini-api/docs
- https://openweathermap.org/api
- https://flask-socketio.readthedocs.io/
- https://www.chartjs.org/docs/
- https://apscheduler.readthedocs.io/
- https://www.geeksforgeeks.org/